2.4 AVALIAÇÃO DE DESEMPENHO

O desempenho de um programa paralelo pode ser afetado por diversos fatores. O primeiro é a granularidade da tarefa, ou seja, a taxa entre o número de bytes recebidos e o número de operações de ponto flutuante efetuadas por um processo.

No caso de um sistema distribuído, o número total de mensagens enviadas também é um fator a ser considerado. Como uma regra geral, enviar um pequeno número de mensagens grandes levará menos tempo do que enviar um grande número de pequenas mensagens. Contudo, isso nem sempre ocorre, já que algumas aplicações podem sobrepor a computação com o envio de pequenas mensagens. O número ideal de mensagens é específico para cada aplicação.

Há aplicações bem adequadas ao paralelismo funcional, enquanto outras se aplicam bem ao paralelismo de dados (Seção 2.2.4). No paralelismo funcional diferentes máquinas fazem diferentes tarefas baseadas nas suas capacidades específicas. No paralelismo de dados, os dados são distribuídos para todas os processos ou *threads*. As operações (frequentemente bastante similares) são realizadas sobre cada conjunto de dados e a informação é transmitida entre os processadores até a resolução final do problema.

Os programas paralelos podem também usar uma combinação de ambos os modelos para explorar totalmente as potencialidades de cada máquina. Computadores diferentes terão capacidades de processamento diferentes. Mesmo que todas as máquinas sejam do mesmo tipo e modelo, elas podem apresentar diferenças de desempenho devido à carga de processamento proveniente dos outros usuários.

Considerações a respeito da rede de interconexão são também importantes caso se esteja utilizando um *cluster* ou um computador similar. Os atrasos da comunicação na rede podem causar problemas. Além disso, a capacidade de processamento disponível pode variar dinamicamente dependendo da carga de trabalho de cada máquina.

Para lidar com esses problemas, alguma forma de balanceamento de carga (Seção 2.3) deve ser utilizada. Em qualquer dos casos, é necessário avaliar corretamente o desempenho de um programa. Algumas considerações sobre métricas de avaliação de desempenho são apresentadas na próxima seção.

2.4.1 Medidas de desempenho

Uma das questões que surgem, ao elaborar um programa paralelo, é saber se o seu desempenho é adequado quando executado em um ambiente paralelo. Deste modo, para avaliar o desempenho de um sistema de processamento paralelo, as métricas mais importantes para serem consideradas são: *speedup* (também conhecido como ganho de desempenho ou aceleração), eficiência e escalabilidade.

Diversos fatores influenciam essas métricas, tais como o custo de sincronização e comunicação, a distribuição das tarefas entre os processadores e o percentual do tempo de execução

do programa passível de paralelização. Com relação a este último item, alguns estudos teóricos auxiliam na previsão do ganho de desempenho máximo a ser obtido com a paralelização de uma aplicação sequencial. São estes: a *Lei de Amdahl* e a *Lei de Gustafson*.

Speedup

O ***speedup***, ou aceleração, mede a razão entre o tempo gasto para execução de um algoritmo ou aplicação em um único processador ($T(1)$) e o tempo gasto na execução com P processadores ($T(P)$), como visto na equação a seguir:

$$S(P) = \frac{T(1)}{T(P)} \tag{2.1}$$

Em condições ideais, quando o *speedup* é sempre igual a P, onde P é o número de processadores em uso, temos o chamado *speedup* linear. Mas, em geral, o *speedup* é menor do que P, devido principalmente à sobrecarga de comunicação entre os diferentes fluxos de execução do programa, perdas por sincronização e decomposição de tarefas mal feita. Quando isso acontece, chamamos o *speedup* de sublinear. Essa situação pode se deteriorar até o ponto no qual a adição de mais processadores diminui, em vez de aumentar, o ganho obtido, caracterizando um "retorno negativo".

Em algumas situações especiais, *speedups* superiores a P podem ser obtidos (denominados "*speedups* superlineares"). Exemplos destas situações são aplicações onde o volume de dados manipulados é grande o suficiente para exceder o tamanho da memória cache de um único processador. Nesse caso, ao dividir a aplicação entre P processadores, o volume de dados manipulado por cada processador é aproximadamente dividido por P, sendo este volume agora pequeno o suficiente para poder ser armazenado integralmente, ou com baixo nível de interferência destrutiva, na memória cache de cada processador. Dentro deste quadro, e respeitadas as condições mencionadas anteriormente, é razoável se esperar um *speedup* superior a P no processo de paralelização, já que o desempenho com um único processador fica muito prejudicado pela baixa taxa de acerto nos acessos à memória cache.

Uma situação análoga ocorre quando são feitas buscas em grandes bases de dados, tais como a busca de dados genômicos realizadas por diversas implementações paralela do programa BLAST. Nesse caso, a quantidade de memória RAM acumulada de cada um dos nós do *cluster* permite a movimentação da base de dados do disco inteiramente para a memória. Isso reduz dramaticamente, portanto, o tempo requerido, por exemplo, para o mpiBLAST percorrer toda a base de dados (CORREA; SILVA, 2012).

O *speedup* superlinear pode ocorrer também, sob determinadas condições, quando da execução de algoritmos de *backtracking* e *branch and bound* paralelos (RAO; KUMAR, 1993; QUINN; DEO, 1986). A Figura 2.14 ilustra diferentes curvas de *speedups*.

Outra situação onde *speedups* superiores a P podem ser produzidos se dá quando a com-

plexidade do problema a ser resolvido é $O(NK)$, onde $K > 1$. Nestes casos, ao se distribuir de forma homogênea os dados da aplicação entre os P processadores no processo de paralelização, o tempo gasto por cada processador para resolver o problema sobre este subconjunto de dados, respeitadas as condições mencionadas anteriormente, é aproximadamente:

$$T(P) = \frac{T(1)}{P \times K} \tag{2.2}$$

onde $T(1)$ é o tempo gasto para se obter a solução do problema com um único processador. Portanto, um *speedup* superlinear é obtido. Este tipo particular de situação, pode ser observado, por exemplo, em aplicações para ordenação de um vetor de números.

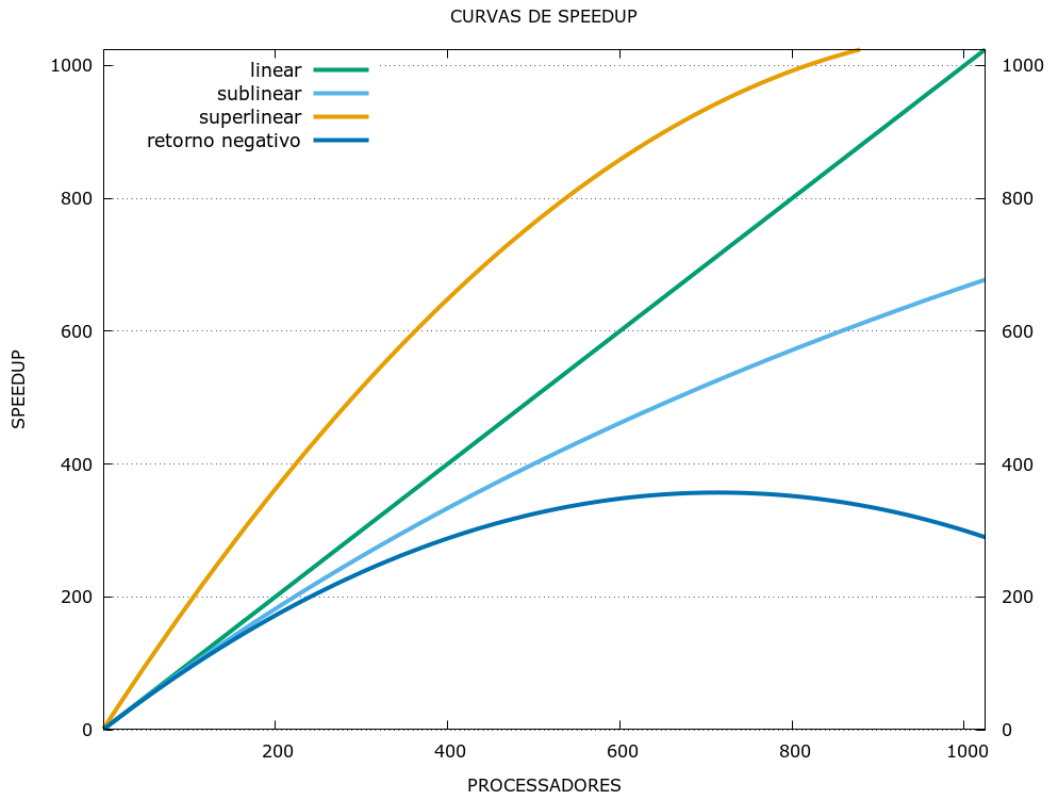

Figura 2.14 – Curvas de *speedup*

Eficiência

A **eficiência** é uma medida de quão efetiva é a adição de novos processadores para ajudar na resolução de um problema. Seu valor é obtido pela razão entre o *speedup* ($S(P)$) e o número de processadores (P) utilizados para obter este *speedup*, conforme podemos observar na seguinte equação:

$$E(P) = \frac{S(P)}{P} \qquad (2.3)$$

Novamente, como o *speedup* em geral é menor do que P por conta da sobrecarga do processamento paralelo, a eficiência tipicamente assume um valor menor do que 1, mas, preferencialmente, próximo de 1. Para se obter *speedup* muito próximo de P e, consequentemente, uma eficiência muito próxima de 1, as seguintes condições devem normalmente ser satisfeitas:

- no código a ser paralelizado, o percentual de código não paralelizável (que continuará sendo executado de forma sequencial) é muito pequeno;
- no processo de paralelização, a distribuição de carga de trabalho entre os P processadores deve ser homogênea;
- os processadores trabalham nos trechos de código executados em paralelo de forma bastante independente, requerendo muito pouca comunicação ou sincronização entre eles.

Escalabilidade

Um sistema é dito escalável quando sua eficiência se mantém constante à medida que o número de processadores P aplicado à solução do problema cresce. Se o tamanho do problema é mantido constante e o número de processadores aumenta, o *overhead* de comunicação tende a crescer e a eficiência a diminuir. Na prática, uma análise da escalabilidade deve considerar a possibilidade de se aumentar proporcionalmente o tamanho do problema a ser resolvido à medida que P cresce de forma a contrabalançar o natural aumento do *overhead* de comunicação quando P cresce.

Considere como exemplo um problema de tamanho S. Usando P processadores esse problema leva um tempo T para ser executado. O sistema é dito escalável se um problema de tamanho $2S$ executado em $2P$ processadores leva o mesmo tempo T. Escalabilidade é frequentemente uma propriedade mais desejável que o *speedup*. Veja maiores detalhes na Seção 2.4.3.

2.4.2 Lei de Amdahl

A lei de Amdahl (AMDAHL, 1967) [4] é usada para determinar o *speedup* teórico máximo de uma aplicação, o qual está limitado pelo tempo necessário para executar a fração sequencial desse programa (a parte do programa não paralelizada). Considerando que a fração de tempo sequencial do programa é conhecida e dada por f_s e que o programa paralelo será executado

[4] Essa lei recebeu o nome depois que Gene Amdahl apresentou-a em um artigo na *AFIPS Spring Joint Computer Conference* em 1967.

em P processadores, então a lei de Amdahl define o *speedup* teórico (S) máximo conforme determinado na equação final a seguir:

$$S(P) = \frac{T(1)}{T(P)} \tag{2.4}$$

$$S(P) = \frac{f_s + (1 - f_s)}{f_s + \frac{(1-f_s)}{P}} \tag{2.5}$$

$$S(P) = \frac{1}{f_s + \frac{(1-f_s)}{P}} = \frac{P}{1 + (P-1)f_s} \tag{2.6}$$

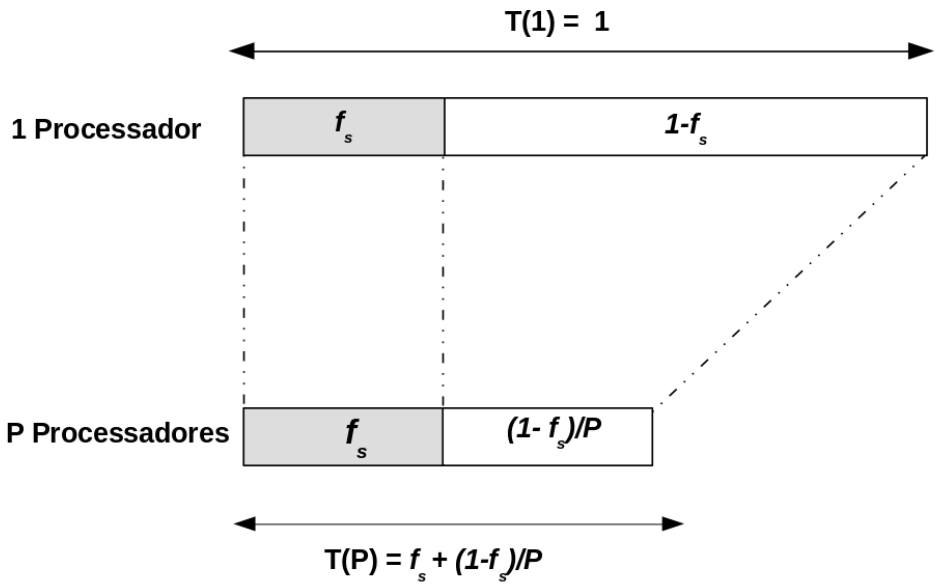

Figura 2.15 – Lei de Amdahl

A Figura 2.15 ilustra sua definição. Podemos verificar que, mesmo com um número infinito de processadores, o *speedup* máximo está limitado a:

$$S(P) \approx \frac{1}{f_s} \tag{2.7}$$

onde f_s é a fração sequencial do programa.

Um programa, por hipótese, demora um total 20 horas quando executado em um único processador, sendo que existe uma fração não paralelizável que leva 1 hora para ser executada.

Então, por mais processadores que sejam utilizados, ao se reduzir a fração paralelizável a um valor perto de zero, o *speedup* máximo estará limitado a 20:

$$S(P) = \frac{1}{\frac{1}{20}} = \frac{1}{0,05} = 20 \qquad (2.8)$$

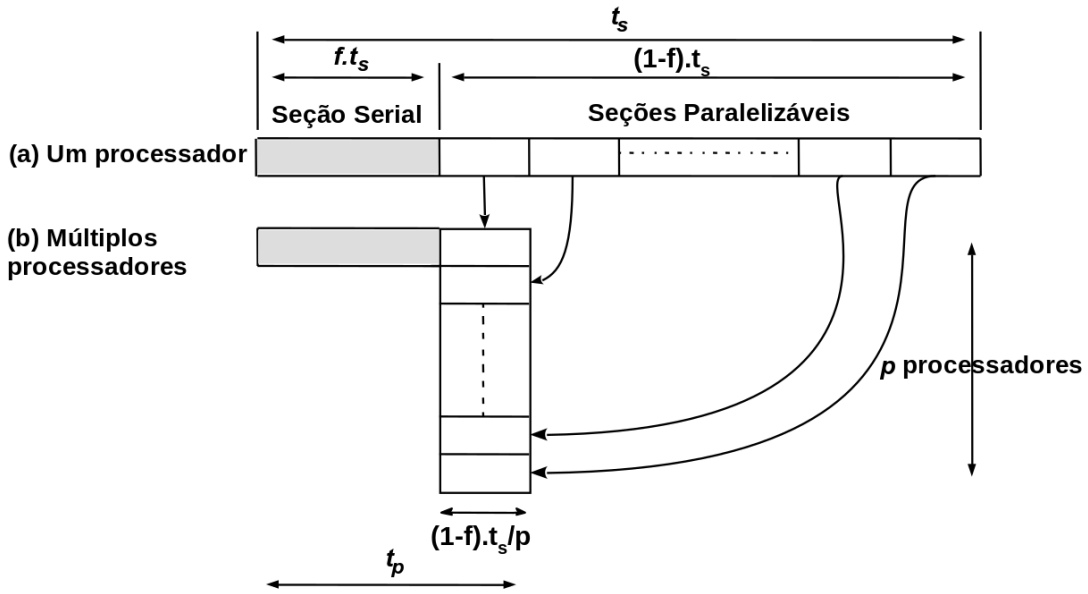

Figura 2.16 – Limite teórico do *speedup* segundo a Lei de Amdahl

A Figura 2.16 [5] apresenta exemplos de limites teóricos para o *speedup* com base na fração não-paralelizável do programa. Com apenas 5% da computação sendo sequencial, observamos um *speedup* teórico máximo de 20, não importando o número de processadores em uso. Se 20% da computação for sequencial, o *speedup* máximo será de 5.

2.4.3 Lei de Gustafson

A lei de Gustafson (GUSTAFSON, 1988), também conhecida como lei de Gustafson–Barsis, afirma que a computação envolvendo um conjunto arbitrariamente grande de dados de entrada pode ser eficientemente paralelizada, oferecendo um contraponto à lei de Amdahl, que toma como base a premissa de um problema de tamanho fixo, independentemente do número de processadores utilizados.

A proposta de Gustafson considera que os programadores tendem a aumentar o tamanho dos problemas à medida que o número de processadores aumenta, de modo a manter fixo o tempo de resolução de um problema. Assim, à medida que mais processadores estão dispo-

[5] Adaptado de (WILKINSON; ALLEN, 1998)

níveis, problemas maiores podem ser resolvidos consumindo a mesma quantidade de tempo, resultando em uma diminuição proporcional da fração sequencial.

Nos dois casos, tanto na Lei de Amdahl como na de Gustafson, assumimos uma distribuição igual da carga de processamento entre todos os processadores.

Segundo Gustafson, o tempo de execução de um programa em um computador paralelo pode ser definido como a soma de duas parcelas:

$$T(P) = a + b \tag{2.9}$$

onde a é o tempo sequencial (f_s) e b é o tempo paralelo ($1 - f_s$), em qualquer um dos processadores P, sem considerar o *overhead*, como pode ser visto na Figura 2.17. Considerando que b, o tempo paralelo por processador, deve ser fixado enquanto P é variado, o tempo correspondente para o processamento sequencial é dado por:

$$T(1) = a + P \cdot b \tag{2.10}$$

O *speedup* é definido, correspondentemente, como:

$$S(P) = \frac{T(1)}{T(P)} = \frac{a + P \cdot b}{a + b} = \frac{f_s + P \cdot (1 - f_s)}{f_s + (1 - f_s)} \tag{2.11}$$

Teremos, por fim:

$$S(P) = f_s + P \cdot (1 - f_s) \tag{2.12}$$

O que nos leva à forma final da lei de Gustafson, resumida na equação seguinte:

$$S(P) = P - f_s \cdot (P - 1) \tag{2.13}$$

onde P é o número de processadores, S é o *speedup* teórico e f_s é a maior fração não-paralelizável do programa.

Então, se f_s é pequeno, o *speedup* é aproximadamente P, como esperado. Pode até mesmo acontecer que f_s diminua à medida que P (junto com o tamanho do problema) aumenta; se isso se verifica, então S se aproxima monotonicamente de P com o crescimento de P.

Por exemplo, se um programa possui uma parte sequencial de tempo constante equivalente a 5% do tempo total da execução paralela, conforme aumenta-se a quantidade de processadores (e o tamanho do problema), o crescimento do *speedup* se mantém. Isso significa que, neste

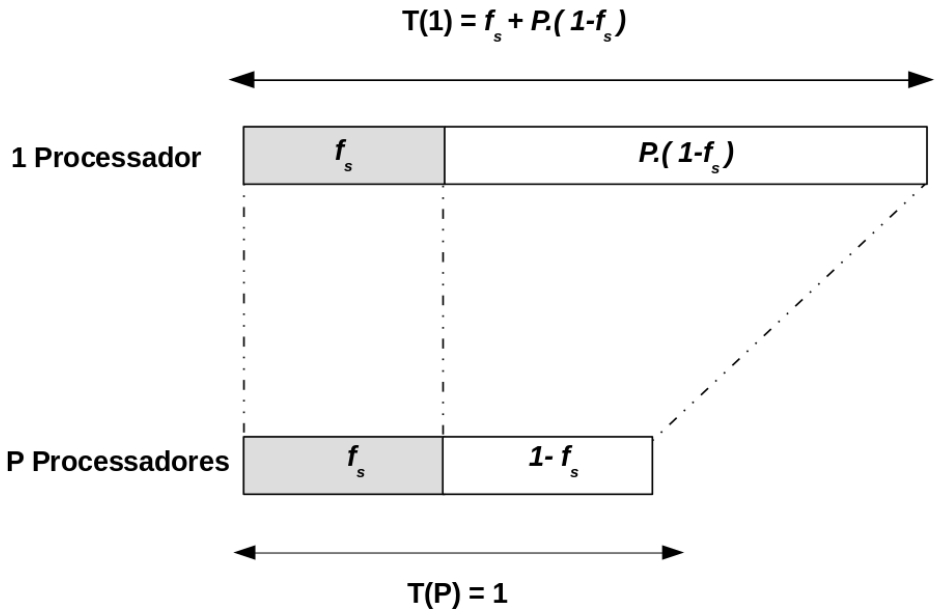

Figura 2.17 – Lei de Gustafson

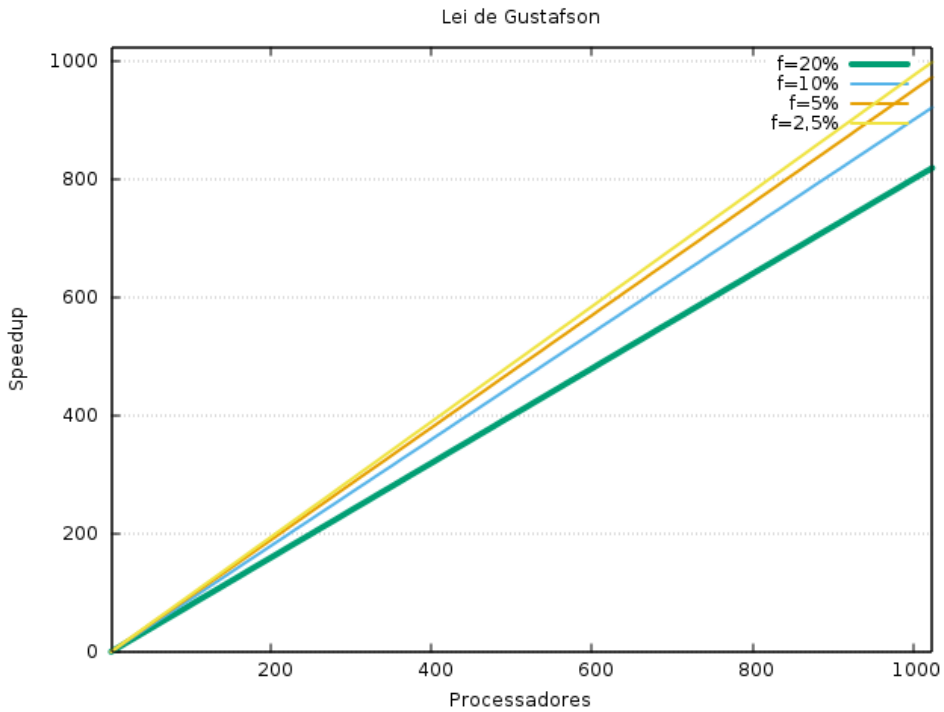

Figura 2.18 – Limite teórico do *speedup* segundo a Lei de Gustafson

exemplo, considerando uma execução com 2 processadores, o *speedup* calculado é de $1,95$; para 20 processadores, o novo *speedup* calculado é de $19,05$; e se aumentar ainda mais a quantidade de processadores, para 200, o *speedup* é de $190,05$. Esses e outros valores podem ser visto no gráfico da Figura 2.18.

Na prática, o resultado final do *speedup* deverá ser algum valor entre a proposta otimista de Gustafson e a pessimista de Amdahl, desde que o programa tenha sido adequadamente elaborado, com balanceamento adequado de carga e baixa carga de sincronização, segundo os métodos já descritos neste capítulo.